Lab 32 — Vulnerability Management Pipeline with Trivy

> **SY0-701 Objective 4.3** — Explain various activities associated with vulnerability management.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~10 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Run the vulnerability management cycle end to end with Trivy: package monitoring of an app's dependencies, an OS scan and a container image scan, CVSS-sorted prioritisation enriched with the CISA KEV threat feed, a time-boxed exception, remediation and a validating rescan.

You'll build: `/root/vm-lab/` containing JSON scan results, a prioritised patch queue, a KEV-matched list, an `exceptions.trivyignore` file with an expiry date and a before/after report.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 31 (vulnerability scanning) introduces the vocabulary; tools installed in step 1.

Background

- **Package monitoring** (software composition analysis) watches the third-party libraries an app depends on. Most application vulnerabilities now arrive through dependencies, not your own code.
- **CVE** identifies one specific vulnerability. **CVSS** (0.0 to 10.0) rates its technical severity. Neither one tells you your **risk**, which also depends on exposure, exploitability and business impact.
- **Threat feeds** add that context: the CISA **Known Exploited Vulnerabilities (KEV)** catalogue lists CVEs exploited in the wild, so a KEV match jumps the queue.
- **Exceptions and exemptions** formally accept a vulnerability for a limited time, with a named risk owner and usually a **compensating control**. An exception without an expiry date is a permanent hole.
- **Validation** means a rescan proves the fix worked. **Reporting** turns the numbers into a trend the business can act on.

Steps

1. Install Trivy

Trivy's official install script downloads a pinned release and verifies its checksum, which is itself a good supply-chain habit.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y curl jq
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh |
sh -s -- -b /usr/local/bin v0.74.0
trivy --version | head -1
mkdir -p /root/vm-lab/app && cd /root/vm-lab
```

2. Package monitoring: scan the app's dependencies

Create a Python app that pins old library versions, then scan the manifest. The first run downloads the vulnerability database.

```
cd /root/vm-lab
cat > app/requirements.txt <<'EOF'
Django==3.2.0
requests==2.25.0
PyYAML==5.3
Jinja2==2.11.2
urllib3==1.26.4
EOF
trivy fs --scanners vuln --severity HIGH,CRITICAL app/
trivy fs --scanners vuln --format json -o deps-before.json app/
```

3. Scan the host operating system

`rootfs` mode inventories every installed OS package and matches it against the vendor's security tracker. This is the credentialed, inside view of the server.

```
cd /root/vm-lab
trivy rootfs --scanners vuln --pkg-types os --severity HIGH,CRITICAL --skip-dirs
/proc,/sys,/root / 2>/dev/null | tail -25
```

4. Scan a container image

Trivy can pull an image straight from the registry without Docker installed. The deliberately old `node:14-alpine` image carries many fixed-but-unpatched vulnerabilities.

```
cd /root/vm-lab
trivy image --image-src remote --severity HIGH,CRITICAL node:14-alpine 2>/dev/null | head
-30
trivy image --image-src remote --format json -o image-before.json node:14-alpine
2>/dev/null
```

5. Build a CVSS-sorted patch queue

Flatten the JSON into one row per vulnerability, sorted by CVSS v3 base score. This is the raw severity view.

```
cd /root/vm-lab
jq -r '.Results[]?.Vulnerabilities[]? | [.VulnerabilityID, .Severity, (.CVSS.nvd.V3Score
// .CVSS.ghsa.V3Score // 0), .PkgName, .InstalledVersion, (.FixedVersion // "no fix")] |
@tsv' \
  deps-before.json image-before.json | sort -u | sort -t$'\t' -k3,3 -gr > patch-queue.tsv
echo "Total findings: $(wc -l < patch-queue.tsv)"
head -10 patch-queue.tsv | column -t -s$'\t'
```

6. Add threat intelligence: the CISA KEV feed

Download the free KEV feed and intersect it with your findings. Anything on KEV is being exploited right now, so it is patched first regardless of the raw score.

```
cd /root/vm-lab
curl -s -o kev.json
https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json
jq -r '.vulnerabilities[].cveID' kev.json | sort -u > kev-ids.txt
echo "KEV catalogue size: $(wc -l < kev-ids.txt)"
cut -f1 patch-queue.tsv | sort -u | grep -Fxf kev-ids.txt > our-kev-matches.txt || true
echo "Our findings that are on KEV: $(wc -l < our-kev-matches.txt)"; cat our-kev-
matches.txt
```

7. Record a time-boxed exception

The image cannot be rebuilt this week, so the risk owner accepts one finding until a fixed date, with a compensating control. The exception lives in its own ignore file that is passed explicitly with `--ignorefile`. Trivy honours the `exp:` date, and the finding automatically reappears after it.

```
cd /root/vm-lab
TOP=$(jq -r '.Results[]?.Vulnerabilities[]? | [(.CVSS.nvd.V3Score // 0), .VulnerabilityID]
| @tsv' image-before.json | sort -gr | head -1 | cut -f2)
cat > exceptions.trivyignore <<EOF
# Exception EX-2026-014 | owner: Head of Platform | compensating control: image runs on an
isolated segment with no inbound internet
$TOP exp:2026-12-31
EOF
cat exceptions.trivyignore
echo "Before exception: $(trivy image --image-src remote --severity HIGH,CRITICAL -q --
format json node:14-alpine | jq '[.Results[]?.Vulnerabilities[]?] | length')"
echo "With exception:   $(trivy image --image-src remote --severity HIGH,CRITICAL -q --
ignorefile exceptions.trivyignore --format json node:14-alpine | jq
'[.Results[]?.Vulnerabilities[]?] | length')"
```

8. Remediate and validate with a rescan

Upgrade the dependencies to fixed versions, rescan, and use `--exit-code` as a pipeline gate so a build with critical findings fails automatically.

```
cd /root/vm-lab
cp app/requirements.txt app/requirements.txt.before
cat > app/requirements.txt <<'EOF'
Django==4.2.20
requests==2.32.4
PyYAML==6.0.2
Jinja2==3.1.6
urllib3==2.5.0
EOF
trivy fs --scanners vuln --format json -o deps-after.json app/
echo "Dependency findings before: $(jq '[.Results[]?.Vulnerabilities[]?] | length' deps-
before.json)  after: $(jq '[.Results[]?.Vulnerabilities[]?] | length' deps-after.json)"
trivy fs --scanners vuln --severity CRITICAL --exit-code 1 -q app/ && echo "GATE PASSED:
no critical dependency findings"
```

9. Scan for secrets and write the report

The same scanner finds hard-coded credentials, which belongs in the same pipeline. Then summarise everything for the asset owner.

```
cd /root/vm-lab
echo 'AWS_ACCESS_KEY_ID="AKIAZ7Q3LABTESTKEY42"' > app/.env
trivy fs --scanners secret app/ 2>/dev/null | grep -E 'AWS|Total' || true
{
  echo "VULNERABILITY MANAGEMENT REPORT - $(date -u '+%Y-%m-%d')"
  echo "Dependencies: $(jq '[.Results[]?.Vulnerabilities[]?] | length' deps-before.json) findings before, $(jq '[.Results[]?.Vulnerabilities[]?] | length' deps-after.json) after upgrade"
  echo "Image node:14-alpine: $(jq '[.Results[]?.Vulnerabilities[]?] | length' image-before.json) findings - action: rebuild on a supported base image"
  echo "KEV matches (patch first): $(wc -l < our-kev-matches.txt)"
  echo "Open exceptions: $(grep -vc '^#' exceptions.trivyignore) (expires 2026-12-31)"
} | tee vm-report.txt
```

Test it

```
cd /root/vm-lab
cat vm-report.txt
trivy fs --scanners vuln --severity CRITICAL --exit-code 1 -q app/; echo "gate exit=$?"
```

Expected: the report shows fewer dependency findings after the upgrade than before (ideally 0), lists the KEV match count and one open exception, and the gate prints `gate exit=0`.

Troubleshooting

Symptom	Fix
`TOOMANYREQUESTS` when pulling the image	Docker Hub rate-limits anonymous pulls per IP. Wait and retry, or use `public.ecr.aws/docker/library/node:14-alpine` as the image name.
`failed to download vulnerability DB`	Retry the command; the DB is pulled from `ghcr.io`. Add `--db-repository public.ecr.aws/aquasecurity/trivy-db` if GHCR is blocked.
The upgraded manifest still shows findings	New CVEs are published all the time. That is the point of continuous package monitoring: bump to the version in the `FixedVersion` column and rescan.

Challenge

Replace the static exception with a policy: write a jq filter that fails the pipeline only if a finding is CRITICAL **or** appears on KEV, and passes everything else into a 30-day patch queue.

Reflection

Two findings sit side by side: a CVSS 9.8 in a library that your code never loads, and a CVSS 7.5 listed on KEV in an internet-facing image. Which do you patch first, and which 4.3 analysis factors justify the decision?

Exam tips

- CVE = the identifier; CVSS = the severity score; NVD = the database that enriches CVEs; KEV = what is actively exploited.
- Exceptions and exemptions need an owner, a compensating control and an expiry. "Insurance" transfers the financial impact, not the vulnerability.
- Package monitoring is listed under application security identification methods. It is SCA, not SAST.
- Exposure factor is the percentage of an asset's value lost in an incident (SLE = AV x EF). It is an analysis input, not a CVSS metric.

References

- Trivy documentation — https://trivy.dev
- CISA Known Exploited Vulnerabilities catalogue — https://www.cisa.gov/known-exploited-vulnerabilities-catalog
- FIRST EPSS (Exploit Prediction Scoring System) — https://www.first.org/epss/
- NIST National Vulnerability Database — https://nvd.nist.gov
- OSV.dev open-source vulnerability database — https://osv.dev

---